

תרגול 12 – Full Stack

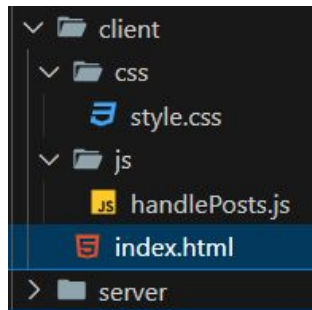
כל הזכויות שמורות לקרן יעקב 2025

התרגול היום

- בתרגול הזה ניקח את צד השרת שעשינו בתרגול הקודם (posts) ונבנה לו צד לקוח פשוט.
- בשביל האחידות, ניקח את הקבצים מהמודל - posts-server.zip
- בקוד ההתחלתי יש את תיקיית server, ניצור לידה תיקיית client ובזמן הפיתוח התיקיות יכולות לשבת אחת ליד השנייה. בהעלאה לשרת - כל אחת תשב במקום אחר.
- נריץ את השרת, נוודא ע"י הדפדפן או פוסטמן שקריאה לשרת עובדת ומביאה את כל הפוסטים מהדטבייס ונתחיל

Index.html – 1

- ניצור עמוד index.html עם דרישות מינימום, אליו נצרף את בוטסטרפ, קובץ CSS שלנו וקובץ js שלנו בשם handlePosts.js
- נוסיף לעמוד header ובו כותרת ראשית



- מתחתיו תהיינה תגיות main, כרגע ללא תוכן ביניהן

Index.html – 1

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
5      <link rel="stylesheet" href="css/style.css"/>
6      <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity=
7      <script src="js/handlePosts.js"></script>
8      <title>Posts list</title>
9    </head>
10   <body>
11     <header>
12       <h1>Posts list</h1>
13     </header>
14     <main></main>
15   </body>
16 </html>
```

posts רשימת 2 - הבאת

- נעבוד בשלבים. בשלב הראשון בפונקציית onload נקרא לפונקציה שמביאים את הפוסטים מהשרת.
- בפונקציה שמביאה את הפוסטים מהשרת נעשה קודם כל הדפסה לקונסול כדי לראות שהכל עובד תקין

```
1 function getPostsListFromServer() {  
2   fetch("http://127.0.0.1:8081/api/posts")  
3     .then(response => response.json())  
4     .then(postsData => console.log(postsData));  
5 }  
6  
7 window.onload = () => {  
8   getPostsListFromServer();  
9 }
```

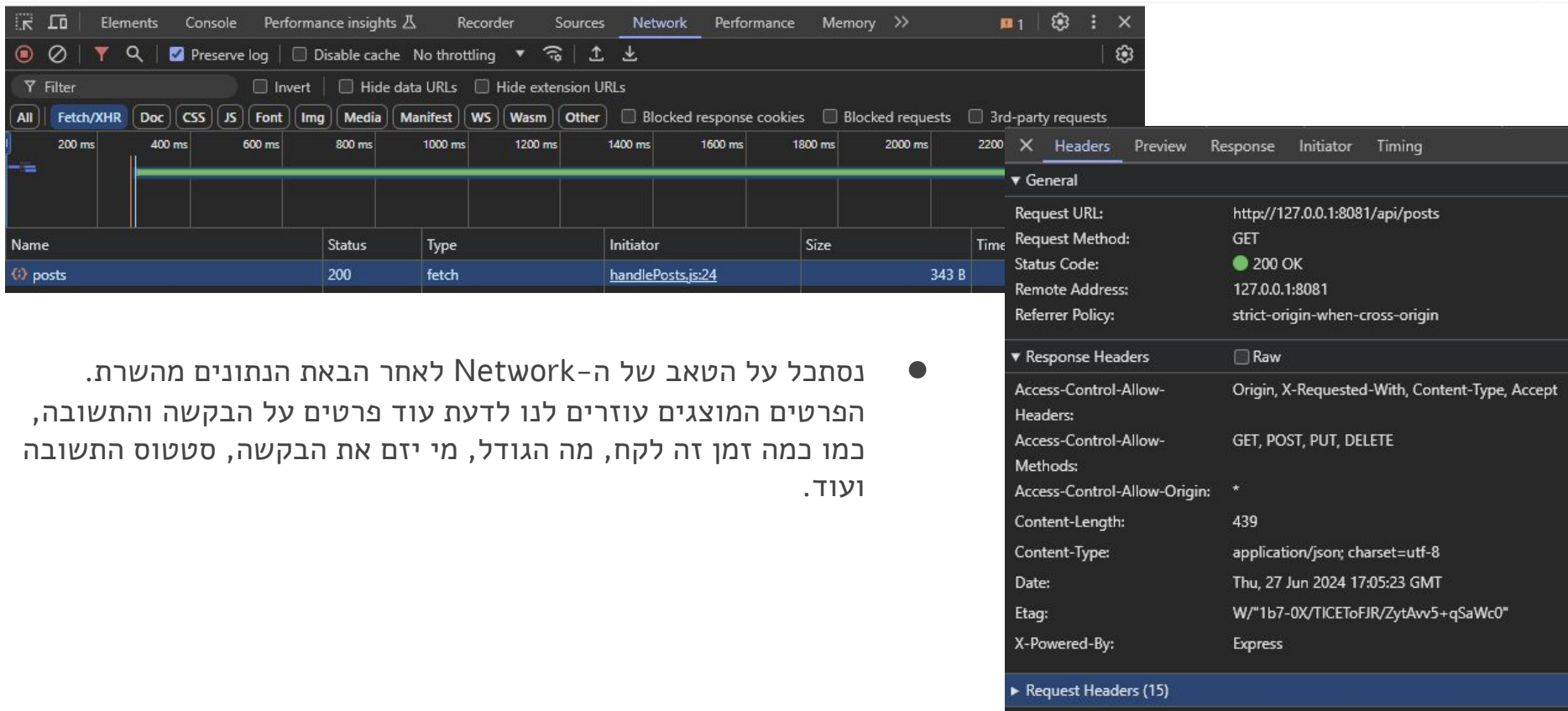
- אחרי שראינו שה json מוחזר לקונסול, נחליף את ההדפסה בפונקציה populatePostsList
- הפונקציה יוצרת ul, מוסיפה לנו קלאס של בוטסטרפ בשם list-group בלולאה קוראת לפונקציה שיוצרת li ומכניסה אותו לרשימה לבסוף מכניסה את הרשימה ל-main

2 - הבאת רשימת posts - המשך

- הפונקציה `createPostItem` יוצרת `li` ומוסיפה לנו קלאס של `list-group-item` בשם `list-group-item` לאחר מכן, עם `innerHTML` היא יוצרת קישור עם `query string` שבו ה-`key` הוא ה-`postId` והערך מגיע מהשרת, והתוכן של ה-`li`, מה שאנחנו רואים על המסך הוא ה-`title` מקף וה-`description`, גם מהשרת (דטבייס)

[illegible]

2 - הבאת רשימת posts - המשך



The screenshot displays the Chrome DevTools Network tab. The top toolbar includes filters for 'Fetch/XHR', 'Doc', 'CSS', 'JS', 'Font', 'Img', 'Media', 'Manifest', 'WS', 'Wasm', and 'Other'. The 'Fetch/XHR' filter is selected. Below the toolbar, a timeline shows a single request labeled 'posts' at approximately 200 ms. The table below the timeline lists the request details:

Name	Status	Type	Initiator	Size	Time
posts	200	fetch	handlePosts.js:24	343 B	

The right-hand pane shows the 'Headers' tab for the selected request. The 'General' section displays the following information:

- Request URL: `http://127.0.0.1:8081/api/posts`
- Request Method: `GET`
- Status Code: `200 OK`
- Remote Address: `127.0.0.1:8081`
- Referrer Policy: `strict-origin-when-cross-origin`

The 'Response Headers' section is expanded, showing the following headers:

- `Access-Control-Allow-Headers`: `Origin, X-Requested-With, Content-Type, Accept`
- `Access-Control-Allow-Methods`: `GET, POST, PUT, DELETE`
- `Access-Control-Allow-Origin`: `*`
- `Content-Length`: `439`
- `Content-Type`: `application/json; charset=utf-8`
- `Date`: `Thu, 27 Jun 2024 17:05:23 GMT`
- `Etag`: `W/"1b7-0X/TICEToFJR/ZytAw5+qSaWc0"`
- `X-Powered-By`: `Express`

The bottom of the pane shows 'Request Headers (15)'.

● נסתכל על הטאב של ה-Network לאחר הבאת הנתונים מהשרת. הפרטים המוצגים עוזרים לנו לדעת עוד פרטים על הבקשה והתשובה, כמו כמה זמן זה לקח, מה הגודל, מי יזם את הבקשה, סטטוס התשובה ועוד.

2 - הבאת רשימת posts - המשך

```
1 function createPostItem(postData) {
2   const post = document.createElement('li');
3   post.classList.add("list-group-item");
4
5   const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;
6   post.innerHTML = postListItem;
7
8   return post;
9 }
```

```
10
11 > function populatePostsList(postsData) {...
12 }
13
14 function getPostsListFromServer() {
15   fetch("http://127.0.0.1:8081/api/posts")
16     .then(response => response.json())
17     .then(postsData => populatePostsList(postsData));
18 }
19
20 window.onload = () => {
21   getPostsListFromServer();
22 }
23 }
```

```
function populatePostsList(postsData) {
  const postsList = document.createElement('ul');
  postsList.classList.add("list-group");

  for (const i in postsData) {
    const post = createPostItem(postsData[i]);
    postsList.appendChild(post);
  }

  document.getElementsByTagName("main")[0].appendChild(postsList);
}
```


3 - הוספת post

- נרצה להוסיף כפתור להוספת פוסט חדש. כרגע ניצור פוסטים עם כותרת ועם תיאור דיפולטיים.
- ב-header, משמאל לכותרת הוסיפו כפתור בוטסטרפ בצבע תכלת. בשביל המיקום העזרו בקובץ ה-CSS שלנו

Add a post	Posts list
Default post title - Default post desc	
Default post title - Default post desc	

- בקובץ ה-js, הוסיפו ב-onload קריאה לפונקציה addListeners
- בפונקציה addListeners נוסף טריגר לחיצה לכפתור שיפעיל את הפונקציה addDefaultPost
- הפונקציה addDefaultPost יוצרת אובייקט js ובו title, description ועושה קריאה לשרת להוספת הפוסט. בסיום קוראת לפונקציה addCreatedPostToList כדי להוסיף את הפוסט לרשימה
- הפונקציה addCreatedPostToList בודקת שהסטטוס תשובה 200 ואם כן תשתמש בפונקציה createPostItem כדי ליצור פוסט חדש ולשים אותו בתוך הרשימה ב-HTML

3 - הוספת post - המשך

Name	X	Headers	Payload	Preview	Response	Initiator	Timing
posts		▼ General					
		Request URL:	http://127.0.0.1:8081/api/posts				
		Request Method:	POST				
		Status Code:	● 200 OK				
		Remote Address:	127.0.0.1:8081				
		Referrer Policy:	strict-origin-when-cross-origin				

X	Headers	Payload	Preview	Response	Initiator	Timing
▼ Request Payload		view source				
▼	{title: "Default post title", description: "Default post desc"}					
	description: "Default post desc"					
	title: "Default post title"					

X	Headers	Payload	Preview	Response	Initiator	Timing
1	true					

● אנו רואים בצילום מסך האחרון שהתשובה המתקבלת מהשרת היא בוליאנית.

מה יכולות להיות הבעיות עם זה?

3 - הוספת post - המשך

```
29 function addCreatedPostToList(response, defaultPostBody) {
30   if (response.status !== 200) {
31     return;
32   }
33
34   const post = createPostItem(defaultPostBody);
35   const postsList = document.querySelector("main > ul");
36   postsList.appendChild(post);
37 }
38
39 function addDefaultPost() {
40   const defaultPostBody = {
41     "title": "Default post title",
42     "description": "Default post desc"
43   }
44
45   fetch("http://127.0.0.1:8081/api/posts", {
46     method: "POST",
47     headers: {
48       "Content-Type": "application/json",
49     },
50     body: JSON.stringify(defaultPostBody)
51   })
52   .then(response => addCreatedPostToList(response, defaultPostBody));
53 }
54
55 function addListeners() {
56   document.getElementById("add-post-button").onclick = () => addDefaultPost();
57 }
58
59 window.onload = () => {
60   getPostsListFromServer();
61   addListeners();
62 }
```

4 - הוספת post - תיקון צד השרת

- התשובה הבוליאנית לא מספיק טובה:
 - אנחנו רוצים לקבל את התשובה ולוודא שהיא הצליחה
 - נרצה להחזיר לצד הלקוח את ה-id החדש של ה-post, למקרה שנצטרך למחוק, לעדכן או להביא אותו
- תקנו את צד השרת, כך שנוודא את התשובה ונחזיר לצד הלקוח את ה-id עם הפרטים החדשים של ה-post

4 - הוספת post - תיקון צד השרת

- שימו לב - יש כאן ערבוב של `async await` עם `promises` כדי להדגים שזה אותו הדבר.
כשאתם תתכנתו - תבחרו שיטה אחת, עדיף `async await`, ושימו לב שלא תהיינה הרבה רמות של אסינכרוניות בקוד, כמו 5 `then` אחד בתוך השני

```
22 // POST localhost:8081/posts
23 async addPost(req, res) {
24   const { dbConnection } = require('../db_connection');
25   const connection = await dbConnection.createConnection();
26   const { body } = req;
27
28   await connection
29     .execute(`INSERT INTO tbl_99_posts (title, description) VALUES ("${body.title}", "${body.description}")`)
30     .then(response => {
31       if (response[0].affectedRows === 1) {
32         getPostById(response[0].insertId)
33           .then(row => res.json(row));
34       }
35       else {
36         // ?
37       }
38     });
39 }
```

4 - הוספת post - תיקון צד השרת

● כתבנו פונקציה שתביא את הפוסט לפי ה-id, איך אפשר לייעל את הקוד בקונטרולר? (שלא יהיה כפול)

```
63  async function getPostById(postId){
64      const { dbConnection } = require('../db_connection');
65      const connection = await dbConnection.createConnection();
66
67      const [rows] = await connection.execute(`SELECT * from tbl_99_posts WHERE id=${postId}`);
68
69      connection.end();
70      return rows[0];
71  }
```

4 - הוספת post - תיקון צד הלקוח

● בצד הלקוח נצטרך לשנות את הקוד כדי לקבל את המידע שהגיע מהבקשה (ולא הבדיקת סטטוס שהיתה קודם)

● את המידע להכניס לפוסט (id לא היה לנו קודם)

```
57 function addCreatedPostToList(response) {
58   const post = createPostItem(response);
59   const postsList = document.querySelector("main > ul");
60   postsList.appendChild(post);
61 }
62
63 function addDefaultPost() {
64   const defaultPostBody = {
65     "title": "Default post title",
66     "description": "Default post desc"
67   }
68
69   fetch("http://127.0.0.1:8081/api/posts", {
70     method: "POST",
71     headers: {
72       "Content-Type": "application/json",
73     },
74     body: JSON.stringify(defaultPostBody)
75   })
76   .then(response => response.json())
77   .then(data => addCreatedPostToList(data));
78 }
```

```
<ul class="list-group"> flex
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> ... </li>
  > <li class="list-group-item"> == $0
    <a href="post.html?postId=62">Default post title - Default post desc</a>
    <button class="btn btn-danger" id="delete-post-button" type="button">delete
    </button>
  </li>
  > <li class="list-group-item">
    <a href="post.html?postId=63">Default post title - Default post desc</a>
    <button class="btn btn-danger" id="delete-post-button" type="button">delete
    </button>
  </li>
</ul>
```

5 - מחיקת post

- ליד כל פוסט נשים כפתור מחיקת פוסט. לשם כך נעזר שוב בבוטסטרפ ונשנה את הפונקציה `createPostItem`
- בפונקציה `createPostItem` ניצור כפתור, נגדיר לו `id`, `type`, את הכיתוב על הכפתור ו-`event listener` בשם `deletePost`. לבסוף נוסיף את הכפתור ל-`post`. כפתור מחיקה אחד משוייך לכל פוסט.
- הפונקציה `deletePost` תעשה את הקריאה לשרת, ע"י `fetch` ותשובה מוצלחת תקרא לפונקציה `deletePostItemFromList`
- הפונקציה `deletePostItemFromList` תבדוק שהסטטוס שהגיע הוא 200, ואם כן תמצא את השורה המתאימה ברשימה ותוריד אותה.

איך עוד יכולנו לממש את זה? מה היתרונות והחסרונות בהבאת הרשימה מחדש?

5 - מחיקת post - המשך

Add a post		Posts list
Default post title - Default post desc		delete
Default post title - Default post desc		delete
Default post title - Default post desc		delete
Default post title - Default post desc		delete
Default post title - Default post desc		delete
Default post title - Default post desc		delete
Default post title - Default post desc		delete

Name	X	Headers	Preview	Response	Initiator	Timing
 26	▼ General					
Request URL:			http://127.0.0.1:8081/api/posts/26			
Request Method:			DELETE			
Status Code:			 200 OK			
Remote Address:			127.0.0.1:8081			
Referrer Policy:			strict-origin-when-cross-origin			
▶ Response Headers (10)						
▶ Request Headers (15)						

5 - מחיקת post - המשך

```
1 function deletePostItemFromList(response, postId) {
2   if (response.status !== 200) {
3     return;
4   }
5
6   const postItemToDelete = document.querySelectorAll(`a[href='post.html?postId=${postId}']`);
7   postItemToDelete[0].parentElement.remove();
8 }
9
10 function deletePost(postId) {
11   fetch(`http://127.0.0.1:8081/api/posts/${postId}`, {
12     method: "DELETE",
13     headers: {
14       "Content-Type": "application/json",
15     }
16   })
17   .then(response => deletePostItemFromList(response, postId));
18 }
19
20 function createPostItem(postData) {
21   const post = document.createElement('li');
22   post.classList.add("list-group-item");
23
24   const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;
25   post.innerHTML = postListItem;
26
27   const deletePostButton = document.createElement('button');
28   deletePostButton.classList.add("btn", "btn-danger");
29   deletePostButton.setAttribute("id", "delete-post-button");
30   deletePostButton.setAttribute("type", "button");
31   deletePostButton.textContent = "delete";
32   deletePostButton.addEventListener("click", () => deletePost(postData.id));
33
34   post.appendChild(deletePostButton);
35
36   return post;
37 }
```

Index.html – 1

```
<!DOCTYPE html>
<html>
  <head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"crossorigin="anonymous">
    <link rel="stylesheet" href="css/style.css"/>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"crossorigin="anonymous"></script>
    <script src="js/handlePosts.js"></script>
    <title>Posts list</title>
  </head>
  <body>
    <header>
      <h1>Posts list</h1>
    </header>
    <main></main>
  </body>
</html>
```

posts רשימת - הבאת

```
function createPostItem(postData) {
  const post = document.createElement('li');
  post.classList.add("list-group-item");

  const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;
  post.innerHTML = postListItem;

  return post;
}

function populatePostsList(postsData) {
  const postsList = document.createElement('ul');
  postsList.classList.add("list-group");

  for (const i in postsData) {
    const post = createPostItem(postsData[i]);
    postsList.appendChild(post);
  }

  document.getElementsByTagName("main")[0].appendChild(postsList);
}

function getPostsListFromServer() {
  fetch("http://127.0.0.1:8081/api/posts")
    .then(response => response.json())
    .then(postsData => populatePostsList(postsData));
}

window.onload = () => {
  getPostsListFromServer();
}
```

post הוספת - 3

```
<body>
  <header>
    <h1>Posts list</h1>
    <button type="button" class="btn btn-info" id="add-post-button">Add a post</button>
  </header>
  <main></main>
</body>
```

3 - הוספת post - המשך

```
function addCreatedPostToList(response, defaultPostBody) {
  if (response.status !== 200) {
    return;
  }

  const post = createPostItem(defaultPostBody);
  const postsList = document.querySelector("main > ul");
  postsList.appendChild(post);
}

function addDefaultPost() {
  const defaultPostBody = {
    "title": "Default post title",
    "description": "Default post desc"
  }

  fetch("http://127.0.0.1:8081/api/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(defaultPostBody)
  })
  .then(response => addCreatedPostToList(response, defaultPostBody));
}

function addListeners() {
  document.getElementById("add-post-button").onclick = () => addDefaultPost();
}

window.onload = () => {
  getPostsListFromServer();
  addListeners();
}
```

4 - הוספת post - תיקון צד השרת

```
async addPost(req, res) {
  const { dbConnection } = require('../db_connection');
  const connection = await dbConnection.createConnection();
  const { body } = req;

  await connection
    .execute(`INSERT INTO tbl_99_posts (title, description) VALUES ("${body.title}", "${body.description}")`)
    .then(response => {
      if (response[0].affectedRows === 1) {
        getPostById(response[0].insertId)
          .then(row => res.json(row));
      }
      else {
        // ?
      }
    });
},

async function getPostById(postId){
  const { dbConnection } = require('../db_connection');
  const connection = await dbConnection.createConnection();

  const [rows] = await connection.execute(`SELECT * from tbl_99_posts WHERE id=${postId}`);

  connection.end();
  return rows[0];
}
```

4 - הוספת post - תיקון צד הלקוח

```
function addCreatedPostToList(response) {
  const post = createPostItem(response);
  const postsList = document.querySelector("main > ul");
  postsList.appendChild(post);
}

function addDefaultPost() {
  const defaultPostBody = {
    "title": "Default post title",
    "description": "Default post desc"
  }

  fetch("http://127.0.0.1:8081/api/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(defaultPostBody)
  })
  .then(response => response.json())
  .then(data => addCreatedPostToList(data));
}
```


5 - מחיקת post

```
function deletePostItemFromList(response, postId) {
  if (response.status !== 200) {
    return;
  }

  const postItemToDelete = document.querySelectorAll(`a[href='post.html?postId=${postId}']`);
  postItemToDelete[0].parentElement.remove();
}

function deletePost(postId) {
  fetch('http://127.0.0.1:8081/api/posts/${postId}', {
    method: "DELETE",
    headers: {
      "Content-Type": "application/json",
    }
  })
  .then(response => deletePostItemFromList(response, postId));
}

function createPostItem(postData) {
  const post = document.createElement('li');
  post.classList.add("list-group-item");

  const postListItem = `<a href='post.html?postId=${postData.id}'>${postData.title} - ${postData.description}</a>`;
  post.innerHTML = postListItem;

  const deletePostButton = document.createElement('button');
  deletePostButton.classList.add("btn", "btn-danger");
  deletePostButton.setAttribute("id", "delete-post-button");
  deletePostButton.setAttribute("type", "button");
  deletePostButton.textContent = "delete";
  deletePostButton.addEventListener("click", () => deletePost(postData.id));

  post.appendChild(deletePostButton);

  return post;
}
```